



# **CS2009**

# **DESIGN AND ANALYSIS OF ALGORITHMS**

**Dr. Hashim Yasin**

**National University of Computer  
and Emerging Sciences,  
Faisalabad, Pakistan.**

# ANALYSIS OF RECURSIVE ALGORITHMS



# Recurrence Relation

3

- A recurrence relation,  $T(n)$ , is a recursive function of an integer variable  $n$ .
- Like all recursive functions, it has one or more recursive cases and one or more base cases.

For example,

$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 2T(n/2) + bn + c & \text{if } n > 1 \end{cases}$$

# Recurrence Relation

4

$$T(n) = \begin{cases} a & \text{if } n = 1 \\ 2T(n/2) + bn + c & \text{if } n > 1 \end{cases}$$

- The portion of the definition that does not contain  $T$  is called the base case of the recurrence relation;
- the portion that contains  $T$  is called the recurrent or recursive case.

# Recurrence Relation

5

- To solve a recurrence relation  $T(n)$  we need to derive a form of  $T(n)$  that is not a recurrence relation.
  - ▣ Such a form is called a **closed form** of the recurrence relation.
- There are five methods to solve recurrence relations that represent the running time of recursive methods:
  - Iteration method (*unrolling and summing*)
  - Substitution method (*Guess the solution and verify by induction*)
  - Recursion tree method
  - Master theorem (Master method)
  - Using Generating functions or Characteristic equations

# Recurrence Relation

6

- To solve a recurrence relation  $T(n)$  we need to derive a form of  $T(n)$  that is not a recurrence relation.
  - ▣ Such a form is called a **closed form** of the recurrence relation.
- We discuss examples with
  - ▣ Decreasing function
  - ▣ Dividing function

# ITERATION METHOD



# Iteration Method

8

## □ Steps:

- **Expand** the recurrence
- Express the **expansion as a summation** by plugging the recurrence back into itself until you see a pattern.
- **Evaluate the summation**

## □ In evaluating the summation one or more of the following summation formulae may be used:

# Review on Summations

- **Constant Series:** For integers  $a$  and  $b$ ,  $a \leq b$ ,

$$\sum_{i=a}^b 1 = b - a + 1$$

- **Linear Series (Arithmetic Series):** For  $n \geq 0$ ,

$$\sum_{i=1}^n i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}$$

- **Quadratic Series:** For  $n \geq 0$ ,

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \cdots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

# Review on Summations

- **Cubic Series:** For  $n \geq 0$ ,

$$\sum_{i=1}^n i^3 = 1^3 + 2^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

- **Geometric Series:** For real  $x \neq 1$ ,

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \cdots + x^n = \frac{x^{n+1} - 1}{x - 1}$$

# Review on Summations

□ **Linear-Geometric Series:** For  $n \geq 0$ , real  $c \neq 1$ ,

$$\sum_{i=1}^n ic^i = c + 2c^2 + \cdots + nc^n = \frac{-(n+1)c^{n+1} + nc^{n+2} + c}{(c-1)^2}$$

# Review on Summations

## Harmonic Series:

$$\sum_{k=1}^n \frac{1}{k} = 1 + \frac{1}{2} + \dots + \frac{1}{n} \approx \ln n$$

## Others:

$$\sum_{k=1}^n \lg k \approx n \lg n$$

$$\sum_{k=0}^{n-1} c = cn.$$

$$\sum_{k=0}^{n-1} \frac{1}{2^k} = 2 - \frac{1}{2^{n-1}}$$

$$\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{k=0}^n k(k+1) = \frac{n(n+1)(n+2)}{3}$$

ITERATION METHOD

EXAMPLE 4

**TOWER OF HANOI**

# Example 4

14

- Form and solve the recurrence relation for the running time of **Tower of Hanoi** and hence determine its big-O complexity:

```
void hanoi(int n, char from, char to, char temp){
    if (n == 1)
        cout << from << " -----> " << to;
    else{
        hanoi(n - 1, from, temp, to);
        cout << from << " -----> " << to;
        hanoi(n - 1, temp, to, from);
    }
}
```

# Example 4

15

```
void hanoi(int n, char from, char to, char temp){  
    if (n == 1)  
        cout << from << " -----> " << to;  
    else{  
        hanoi(n - 1, from, temp, to);  
        cout << from << " -----> " << to;  
        hanoi(n - 1, temp, to, from);  
    }  
}
```

- The recurrence relation for running time of the method is:

$$T(1) = a \quad \text{if } n = 1$$

$$T(n) = 2T(n - 1) + b \quad \text{if } n > 1$$

Decreasing function

# Example 4

$$\begin{array}{ll} T(1) = a & \text{if } n = 1 \\ T(n) = 2T(n-1) + b & \text{if } n > 1 \end{array}$$

16

**Expanding:**

$$T(1) = a \quad (1)$$

$$T(n) = 2T(n-1) + b \quad \text{if } n > 1 \quad (2)$$

$$= 2[2T(n-2) + b] + b = 2^2 T(n-2) + 2b + b$$

by substitution  $T(n-1)$  in (2)

$$= 2^2 [2T(n-3) + b] + 2b + b = 2^3 T(n-3) + 2^2 b + 2b + b$$

by substitution  $T(n-2)$  in (2)

$$= 2^3 [2T(n-4) + b] + 2^2 b + 2b + b = 2^4 T(n-4) + 2^3 b + 2^2 b + 2^1 b + 2^0 b$$

by substitution  $T(n-3)$  in (2)

$$= \dots\dots$$

$$= 2^k T(n-k) + b[2^{k-1} + 2^{k-2} + \dots + 2^1 + 2^0]$$

# Example 4

$$\begin{aligned} T(1) &= a && \text{if } n = 1 \\ T(n) &= 2T(n-1) + b && \text{if } n > 1 \end{aligned}$$

17

**Expanding:**

$$= 2^k T(n-k) + b[2^{k-1} + 2^{k-2} + \dots + 2^1 + 2^0]$$

$$\begin{aligned} &= 2^k T(n-k) + b \sum_{i=0}^{k-1} 2^i \\ &= 2^k T(n-k) + b(2^k - 1) \end{aligned}$$

Geometric series

$$\begin{aligned} \sum_{k=0}^n x^k &= 1 + x + x^2 + \dots + x^n = \frac{x^{n+1} - 1}{x - 1} \\ &= \frac{2^{k-1+1} - 1}{2 - 1} = 2^k - 1 \end{aligned}$$

□ The base case is reached when  $n - k = 1 \rightarrow k = n - 1$ , we then have:

$$\begin{aligned} T(n) &= 2^{n-1} T(1) + b(2^{n-1} - 1) \\ &= (a + b) 2^{n-1} - b \\ &= \left(\frac{a+b}{2}\right) 2^n - b \end{aligned}$$

Therefore,  
The method **hanoi** is  **$O(2^n)$**

# RECURSION TREE METHOD



# Recursion Tree

19

- Solving recurrences through **tree**
  - ▣ Expanding the recurrence into a tree
  - ▣ Tree nodes represent costs incurred at various levels of the recursion.
  - ▣ Sum up the cost at each level
  - ▣ Difference between depth and height of node
  
- A *recursion tree* is useful for visualizing what happens when a recurrence is iterated.

# Tree ... Terminologies

20

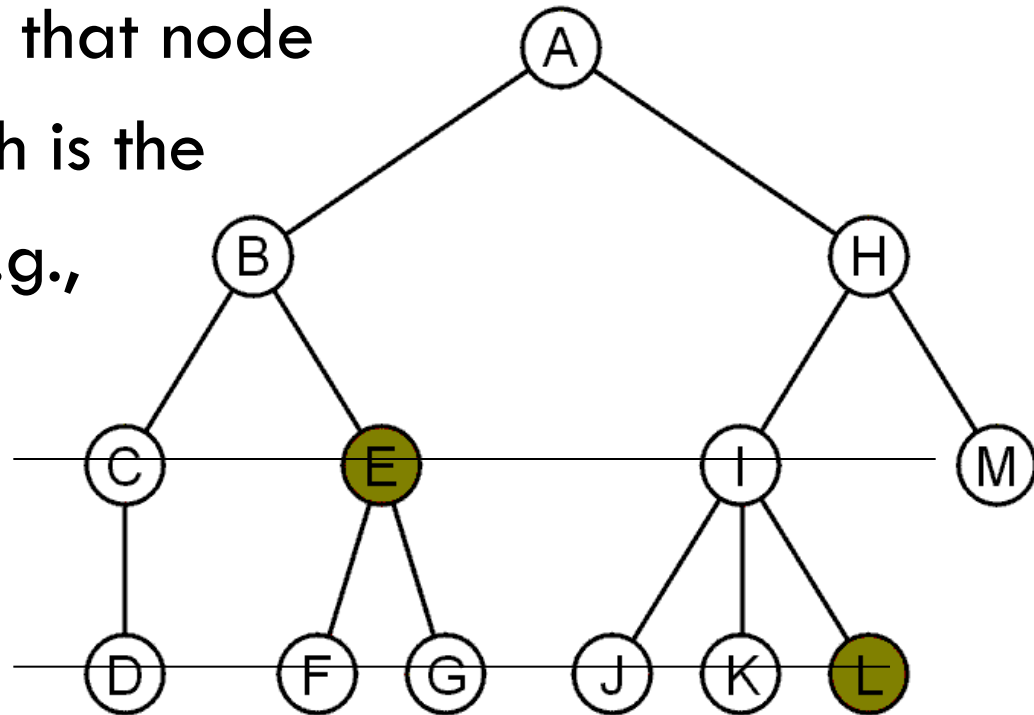
## Depth of the node

- For each node in a tree, there exists a unique path from the root node to that node
- The length of this path is the

**depth of the node**, e.g.,

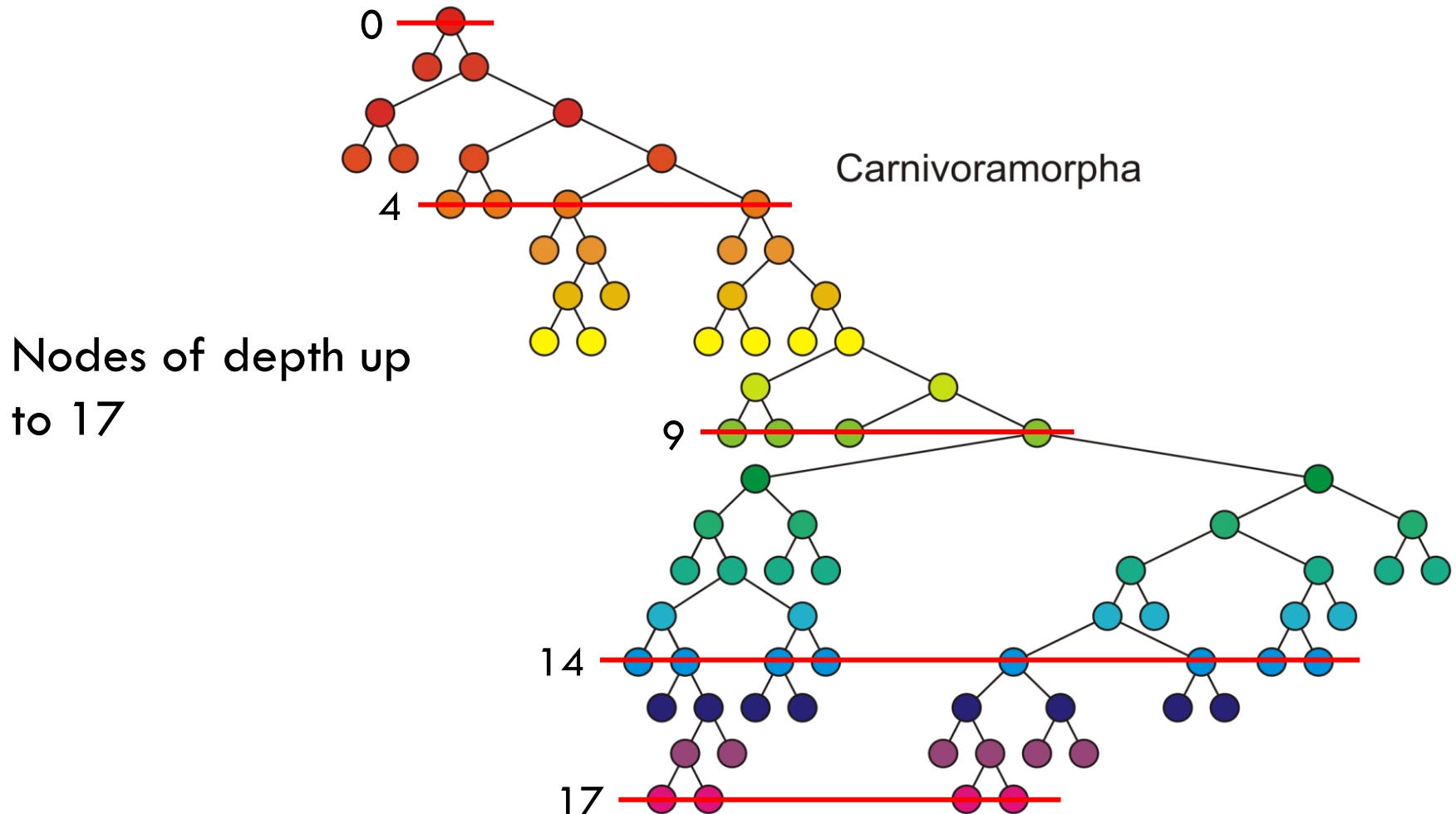
■ E has depth 2

■ L has depth 3



# Tree ... Terminologies

21



Wesley-Hunt, G. D.; Flynn, J. J. "Phylogeny of the Carnivora: basal relationships among the Carnivoramorpha, and assessment of the position of 'Miacoidea'"

# Tree ... Terminologies

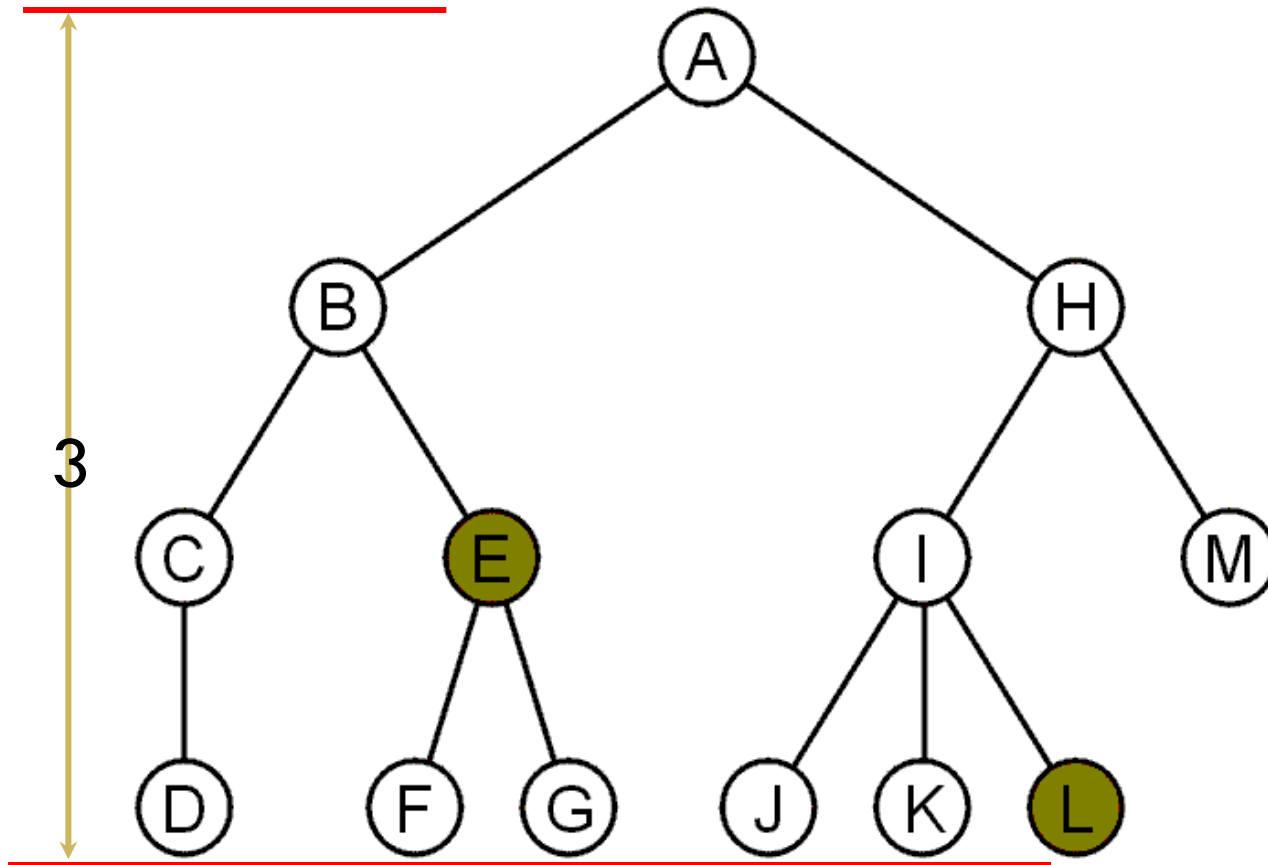
22

- The height of a tree is defined as the *maximum depth* of any node within the tree
- The height of a tree with one node is 0
  - ▣ Just the root node
- For convenience, we define the height of the empty tree to be  $-1$

# Tree ... Terminologies

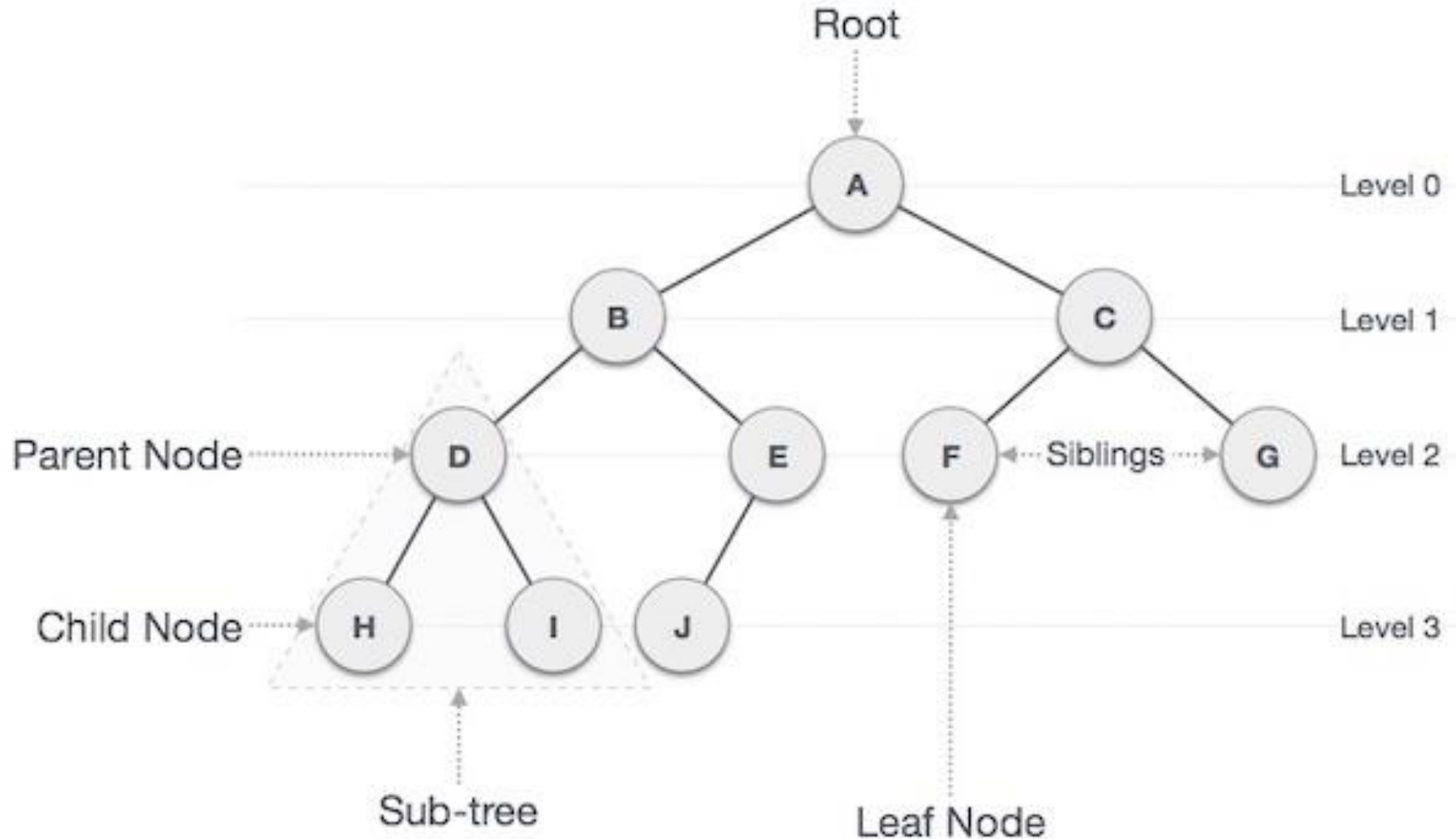
23

Height of the tree is 3



# Tree ... Terminologies

24



# RECURSION TREE DECREASING FUNCTION EXAMPLE 1



# Recursion Tree

26

## Example 1:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
        f(n-1);  
    }  
}
```

□ Therefore, the recurrence relation is:

Decreasing function

$T(0) = a$  for some constant  $a$

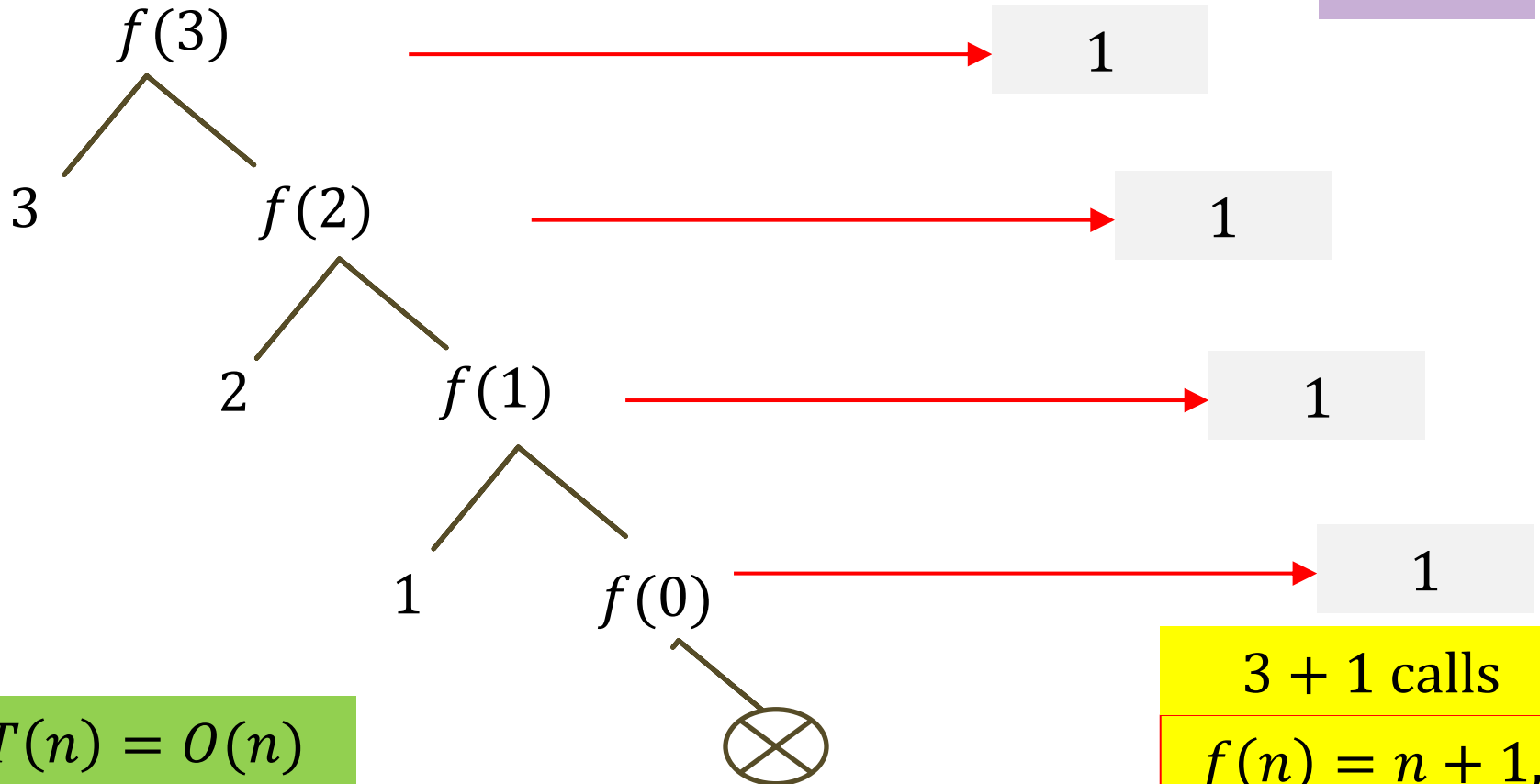
$T(n) = c + T(n - 1)$  for some constant  $c$

# Recursion Tree

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
        f(n-1);  
    }  
}
```

$$T(n) = T(n - 1) + c,$$

$$n = 3$$



# RECURSION TREE DECREASING FUNCTION EXAMPLE 2



# Recursion Tree

29

## Example 2:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        for (i = 0; i < n; i++){  
            cout << n << endl;  
        }  
        f(n-1);  
    }  
}
```

$T(n)$   
 $c$   
 $n+1$   
 $n$   
 $T(n-1)$

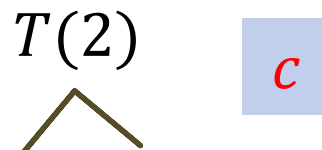
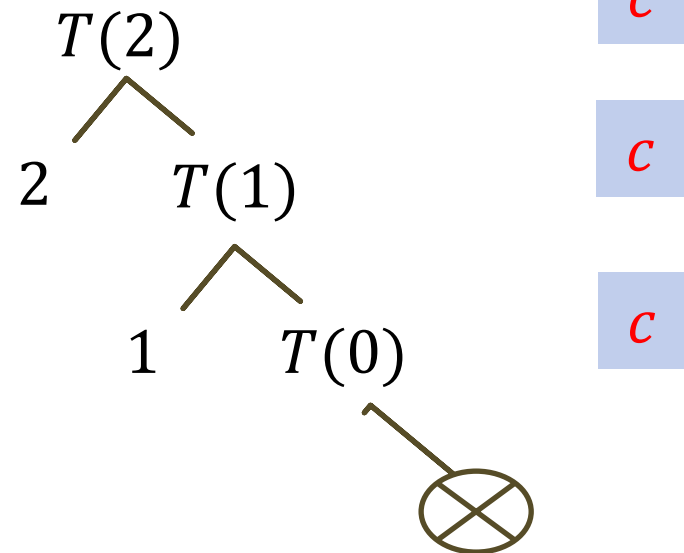
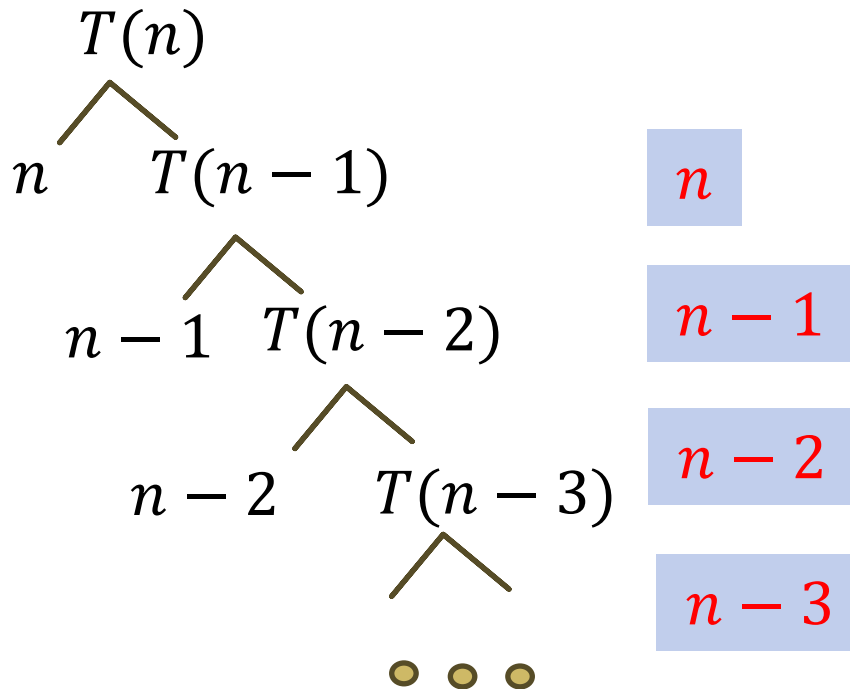
$T(n) = c$  if  $n = 0$   
 $T(n) = T(n-1) + n$  if  $n > 0$

$T(n) = T(n-1) + 2n + c$

Decreasing function

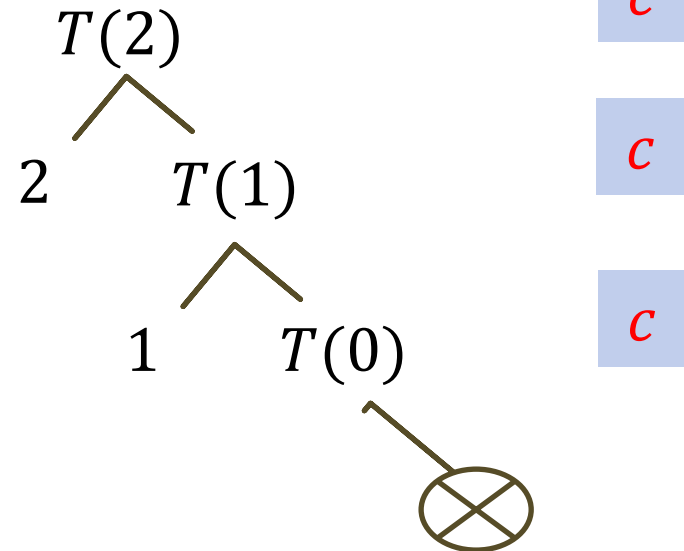
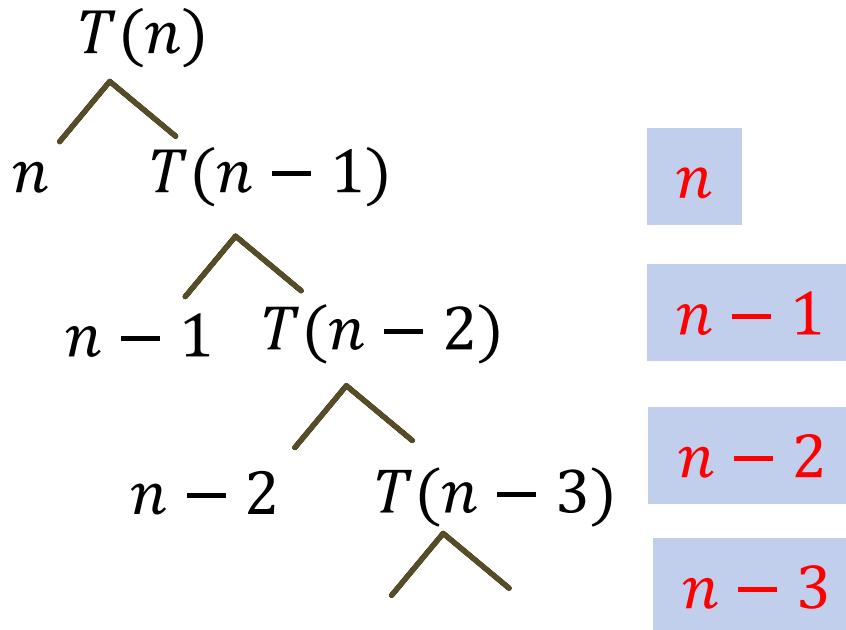
# Recursion Tree

$$T(n) = T(n - 1) + n,$$



# Recursion Tree

$$T(n) = T(n - 1) + n,$$



$$0 + 1 + 2 + \dots + (n - 3) + (n - 2) + (n - 1) + n = \frac{n(n + 1)}{2}$$

$$T(n) = O(n^2)$$

# RECURSION TREE

## DECREASING FUNCTION

### EXAMPLE 3



# Recursion Tree

33

## Example 3:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        for (i = 0; i < n; i=i*2) {  
            cout << n << endl;  
        }  
        f(n-1);  
    }  
}
```

$T(n)$

$\log(n)$

$T(n-1)$

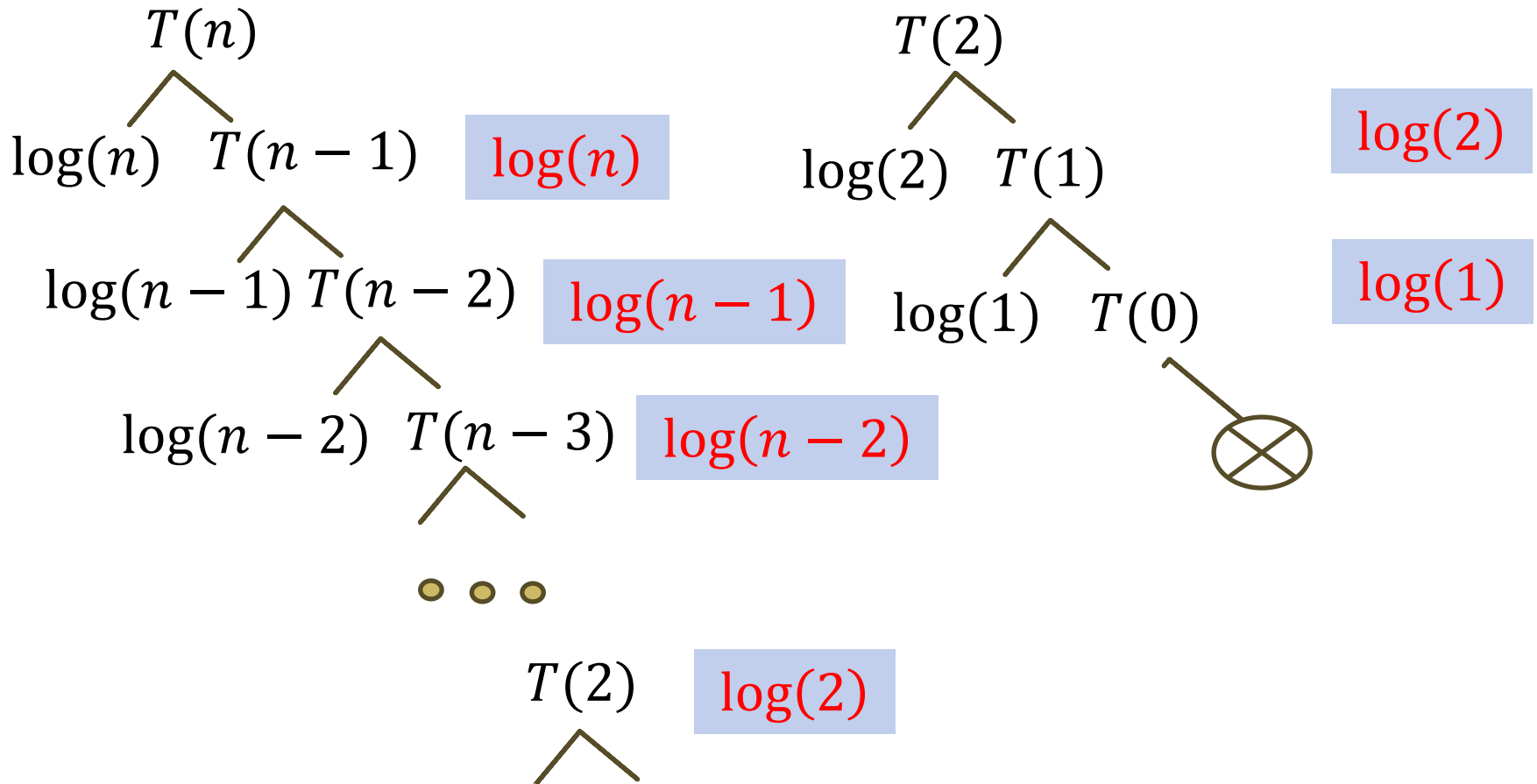
$$\begin{array}{ll} T(n) = c & \text{if } n = 0 \\ T(n) = T(n-1) + \log(n) & \text{if } n > 0 \end{array}$$

$$T(n) = T(n-1) + \log(n) + c$$

Decreasing function

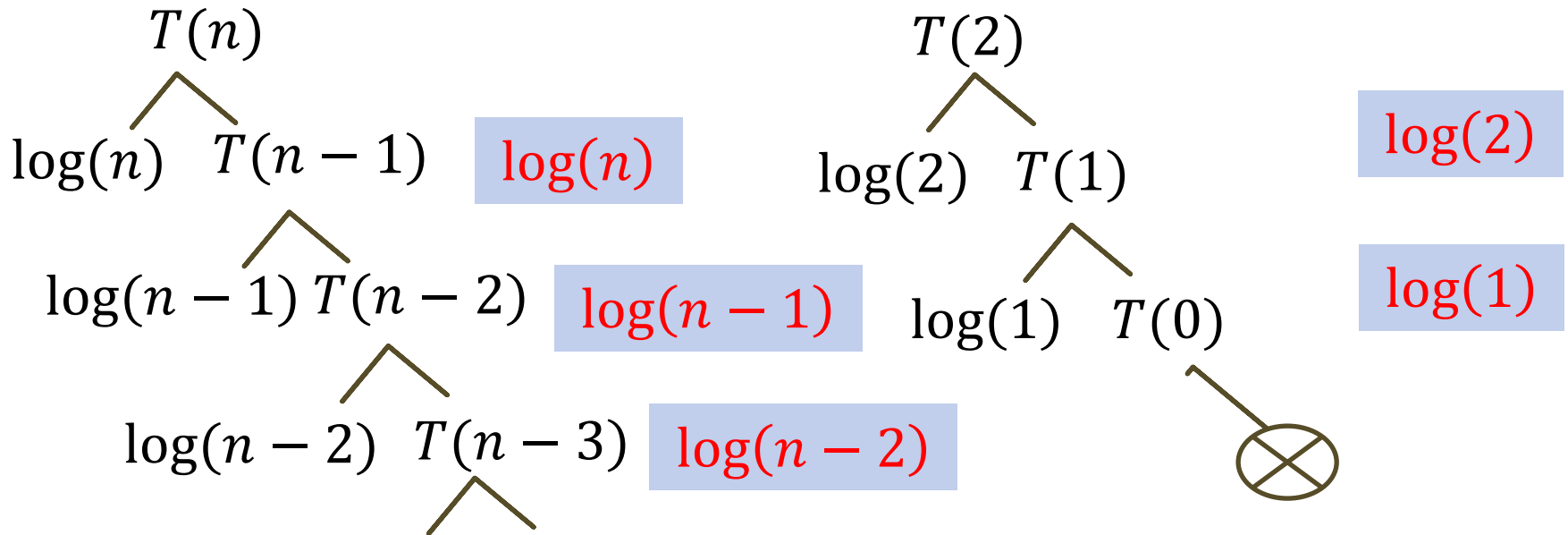
# Recursion Tree

$$T(n) = T(n - 1) + \log(n),$$



# Recursion Tree

$$T(n) = T(n - 1) + \log(n),$$



$$\begin{aligned} &0 + \log(1) + \log(2) + \cdots + \log(n - 2) + \log(n - 1) + \log(n) \\ &= \log[1 \times 2 \times \cdots \times (n - 2) \times (n - 1) \times n] = \log n! \end{aligned}$$

$$T(n) = O(n \log n)$$

# RECURSION TREE DECREASING FUNCTION EXAMPLE 4



# Recursion Tree

37

## Example 4:

Write the recurrence relation for the following method:

```
void f (int n) {  
    if (n > 0) {  
        cout << n << endl;  
    }  
    f(n-1);  
    f(n-1);  
}
```

$T(n)$

$c$

$T(n-1)$

$T(n-1)$

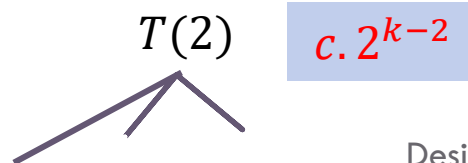
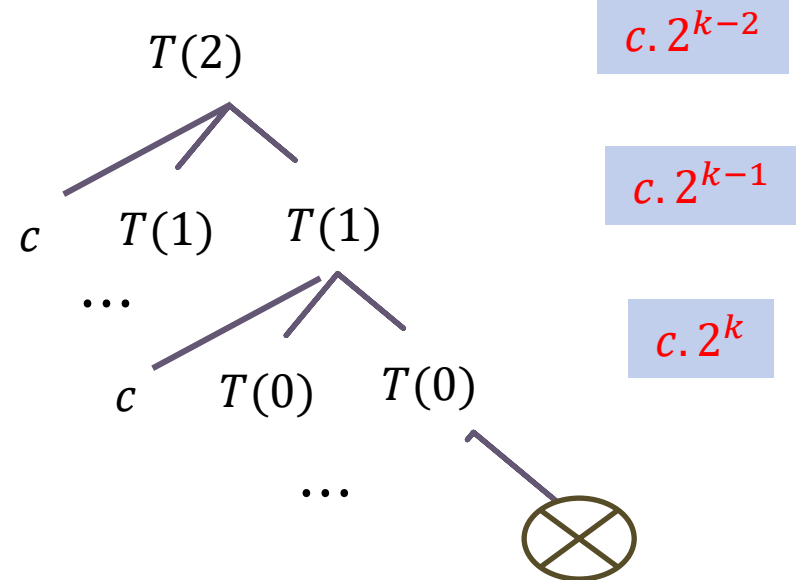
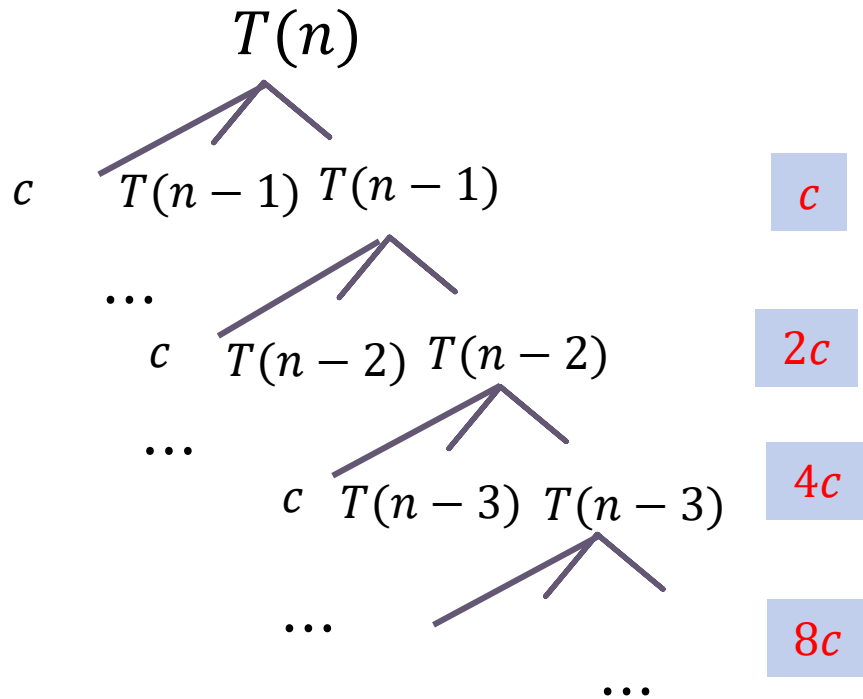
$$\begin{aligned} T(n) &= a & \text{if } n = 0 \\ T(n) &= 2T(n-1) + c & \text{if } n > 0 \end{aligned}$$

$$T(n) = 2T(n-1) + c$$

Decreasing function

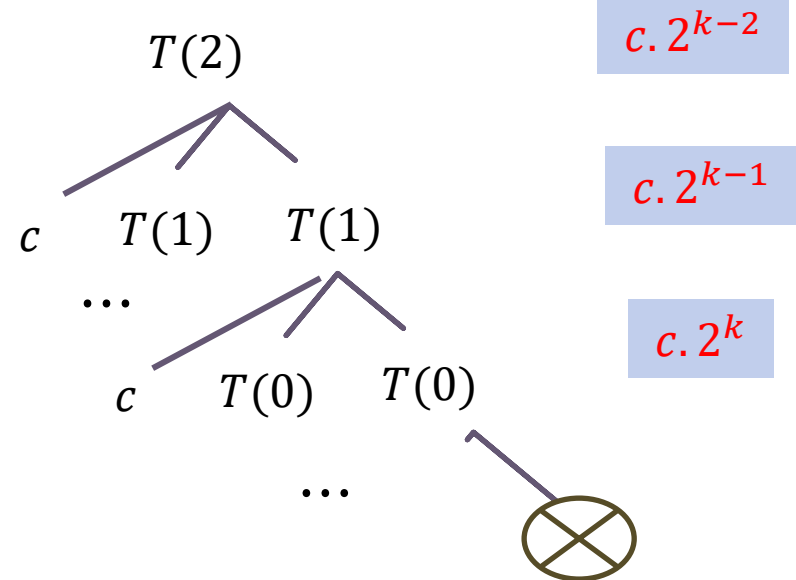
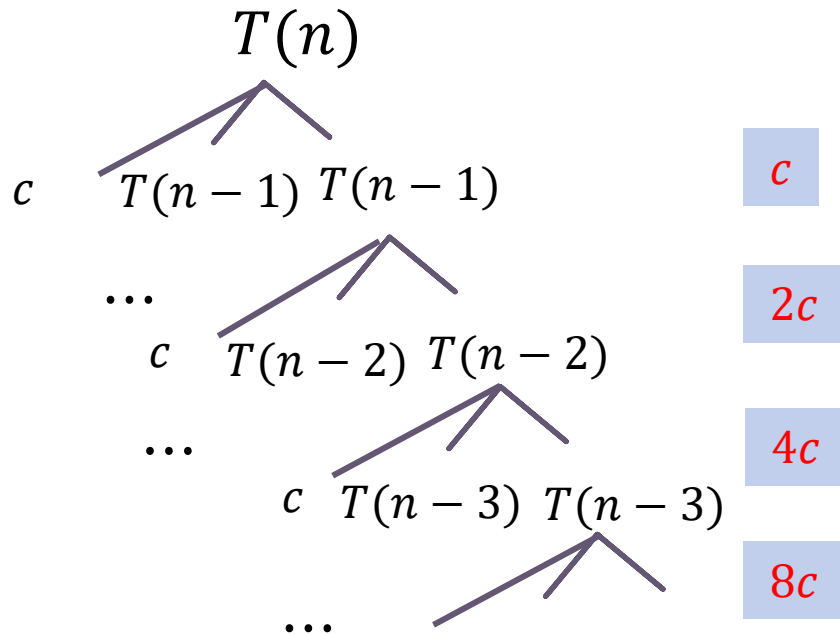
# Recursion Tree

$$T(n) = 2T(n-1) + c,$$



# Recursion Tree

$$T(n) = 2T(n-1) + c,$$



Geometric series

$$\sum_{k=0}^n x^k = 1 + x + x^2 + \dots + x^k = \frac{x^{k+1} - 1}{x - 1} = \frac{2^{k+1} - 1}{2 - 1} = 2^{k+1} - 1$$

$$T(n) = O(2^n)$$

# RECURSION TREE

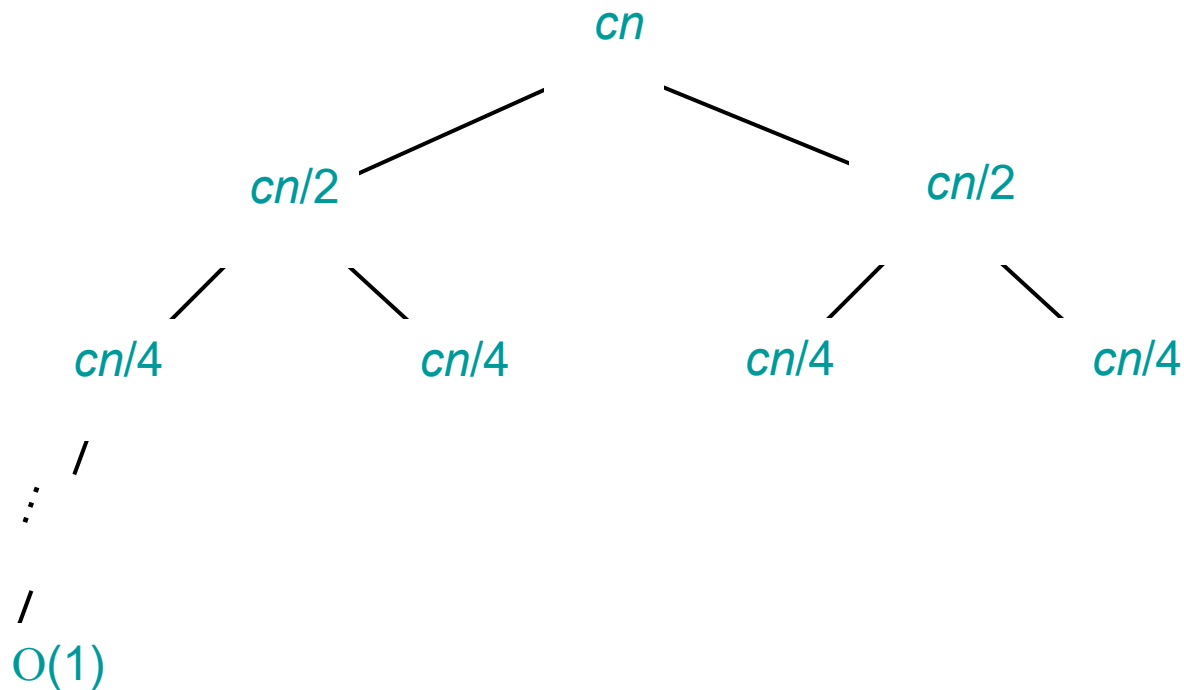
## DIVIDING FUNCTION

### EXAMPLE 5



# Recursion Tree

Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



# Determining depth/height of tree

$$\text{Level } 0 = \frac{n}{(2)^0}$$

$$\text{Level } 1 = \frac{n}{(2)^1}$$

$$\text{Level } 2 = \frac{n}{(2)^2}$$

-----  
-----

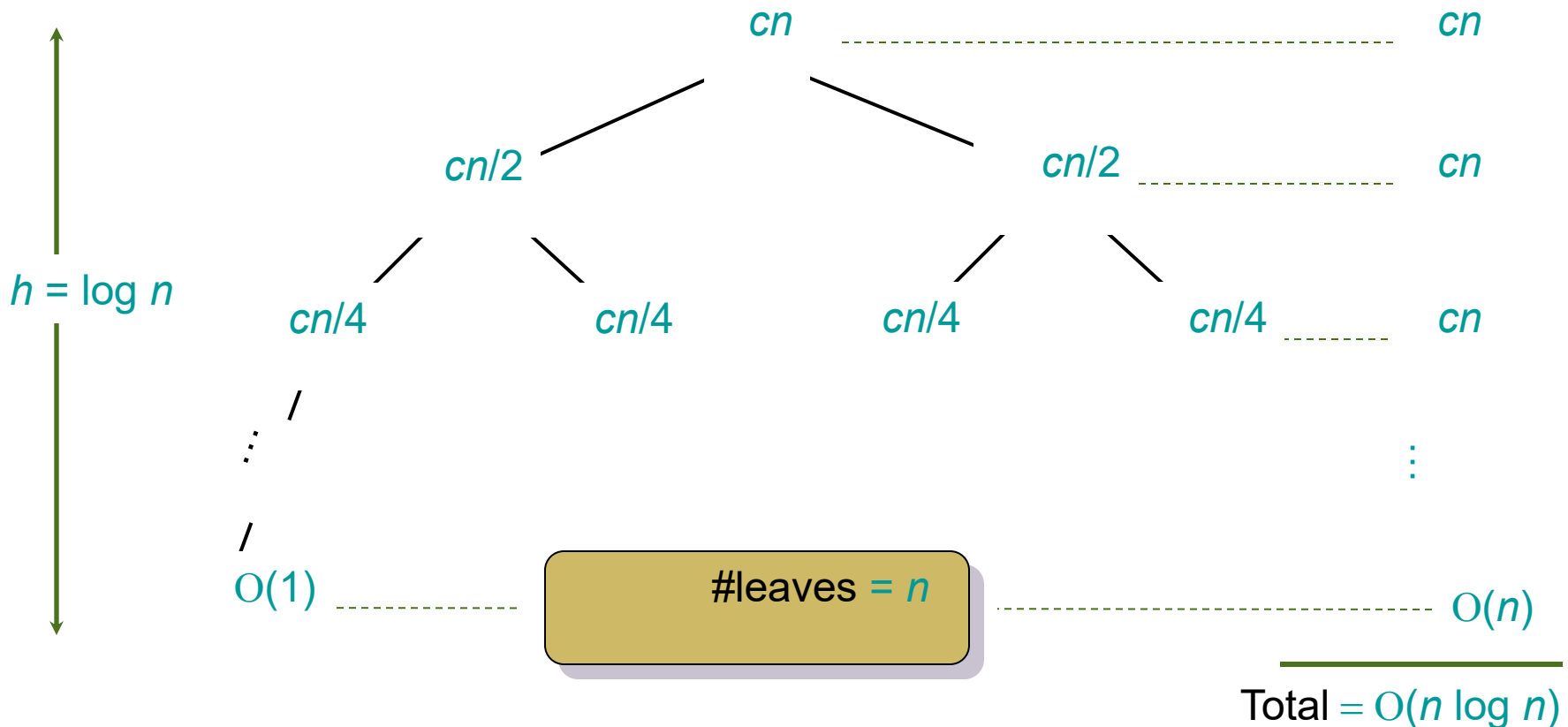
$$\text{Level } i = \frac{n}{(2)^i}$$

$$\text{Level } h = \frac{n}{(2)^h}$$

$$T(1) = T\left(\frac{n}{(2)^h}\right) \Rightarrow \frac{n}{(2)^h} = 1 \Rightarrow n = (2)^h \Rightarrow h = \log_2 n$$

# Recursion Tree

Solve  $T(n) = 2T(n/2) + cn$ , where  $c > 0$  is constant.



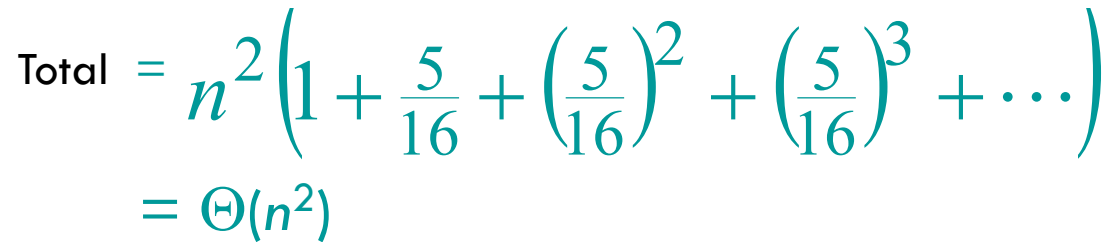
# RECURSION TREE

## DIVIDING FUNCTION

### EXAMPLE 6



Solve  $T(n) = T(n/4) + T(n/2) + n^2$ :



The depth of the tree in this case does not matter.

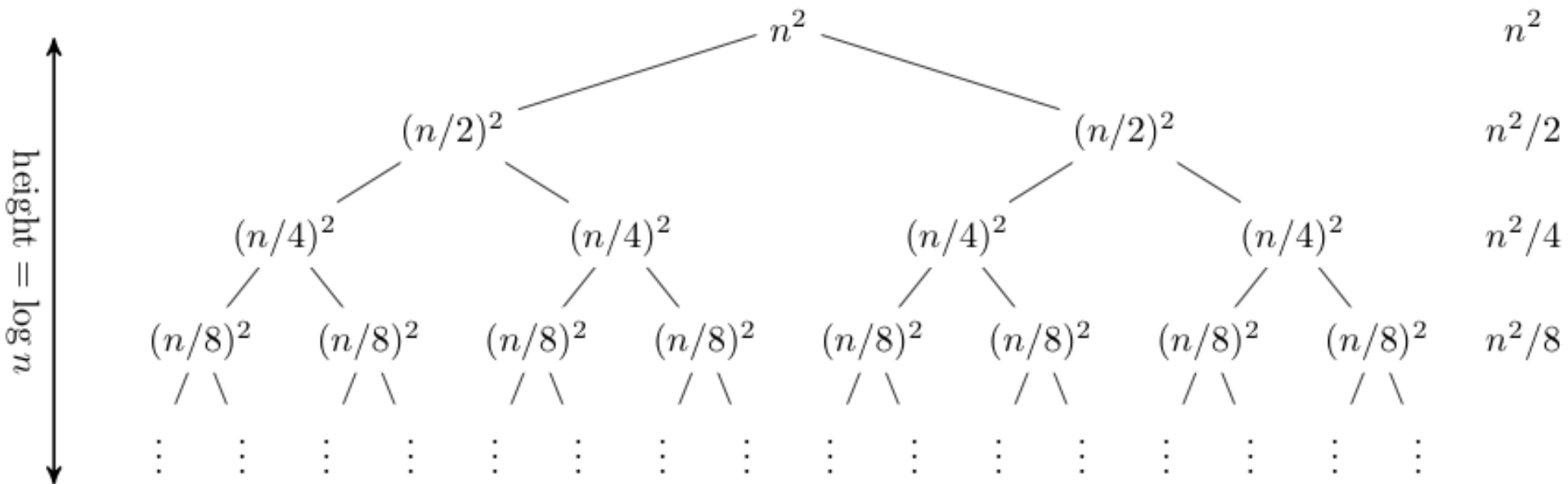
# RECURSION TREE DIVIDING FUNCTION EXAMPLE 7



# Recursion Tree

47

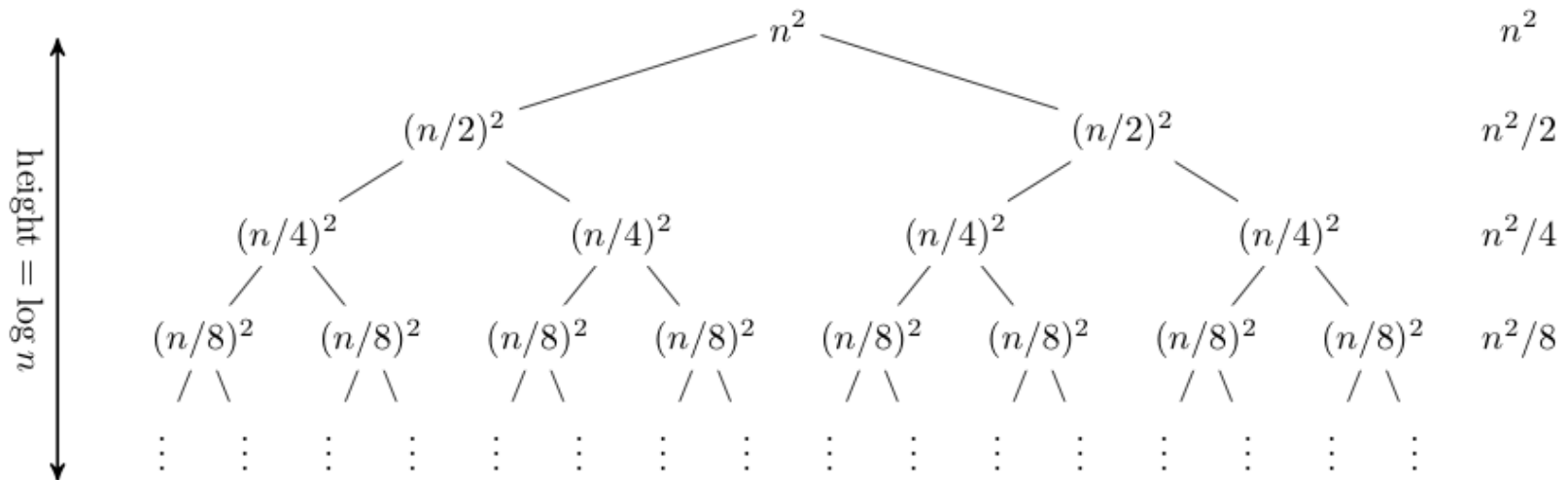
$$T(n) = 2T(n/2) + n^2$$



# Recursion Tree

48

$$T(n) = 2T(n/2) + n^2$$



- This is a geometric series, thus in the limit the sum is  $O(n^2)$ .
- The depth of the tree in this case does not really matter;
  - the amount of work at each level is decreasing so quickly that the total is only a constant factor more than the root.

# Acknowledgement

49

## **Introduction to Algorithms**

Thomas H. Cormen, Charles E. Leiserson,  
Ronald L. Rivest, and Clifford Stein